

System Architecture Design

Introduction

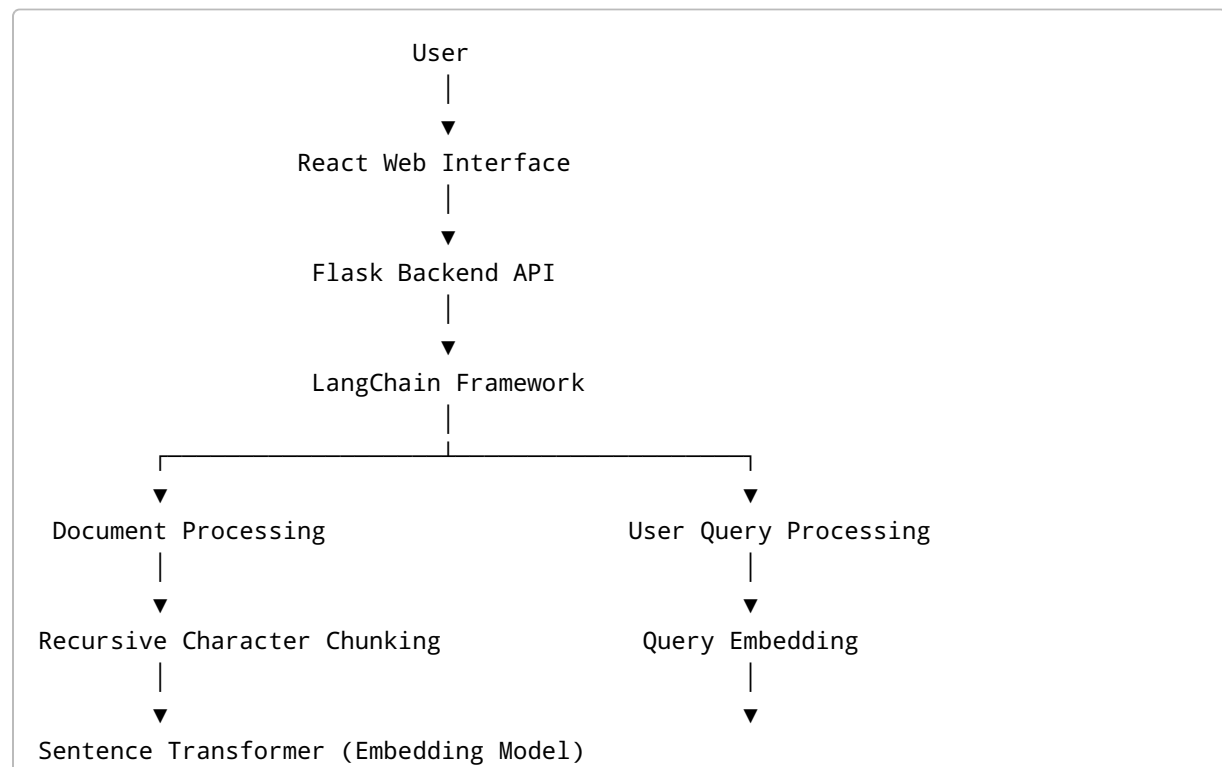
The proposed system is designed as a Retrieval-Augmented Generation (RAG)-based AI application that delivers accurate, context-aware, and reliable responses to user queries. Unlike traditional Large Language Model (LLM) applications that rely solely on pre-trained knowledge, the proposed architecture integrates external knowledge retrieval, semantic search, and automated evaluation frameworks to improve response quality and reduce hallucinations.

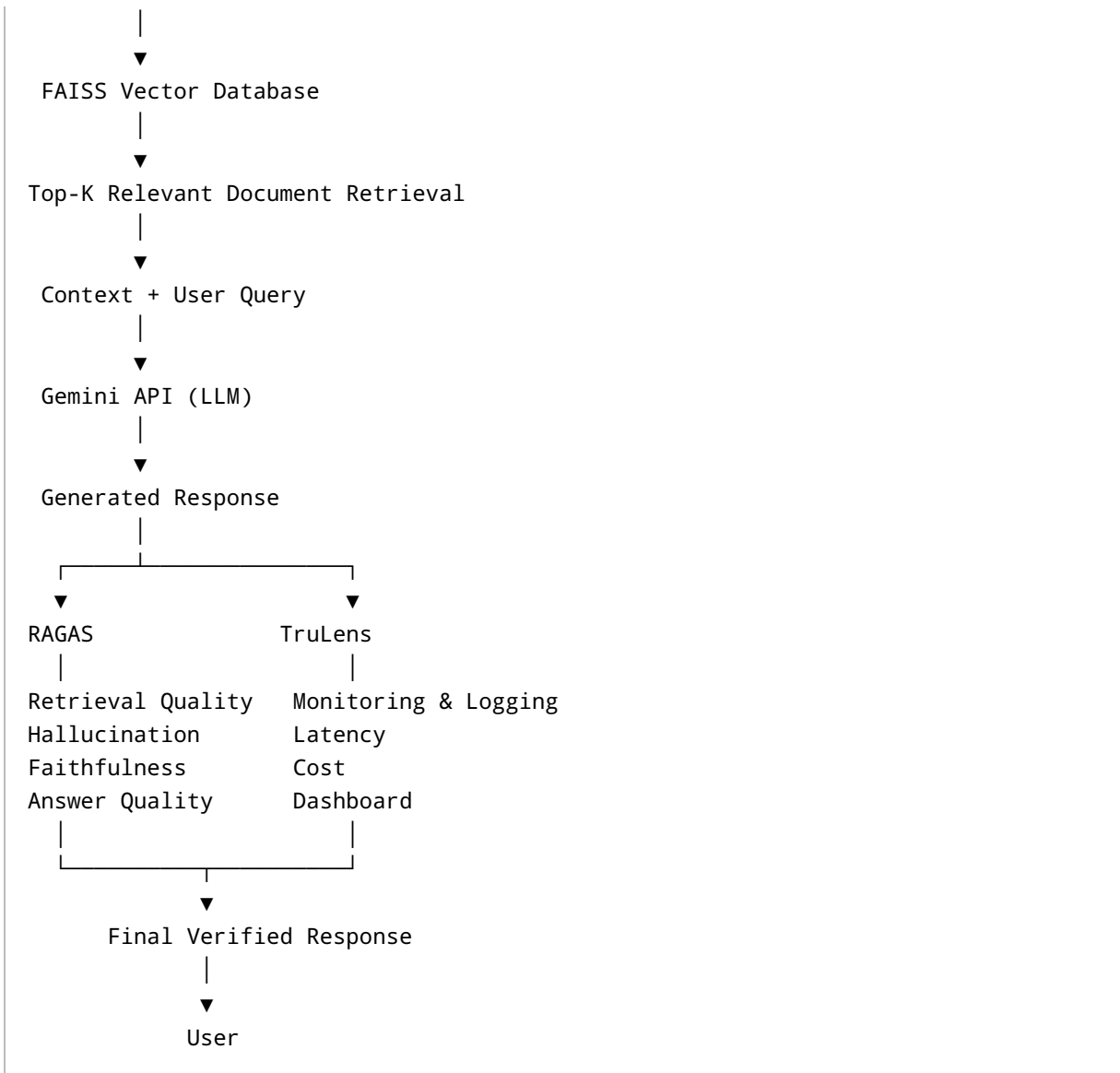
The system utilizes the **Gemini API** as the Large Language Model, **Sentence Transformers (all-MiniLM-L6-v2)** for semantic embeddings, **FAISS** as the vector database, **LangChain** for orchestrating the RAG pipeline, and both **RAGAS** and **TruLens** for evaluating and monitoring the generated responses.

The architecture follows a modular design, making the system scalable, maintainable, and easy to enhance in future development phases.

1. High-Level Architecture

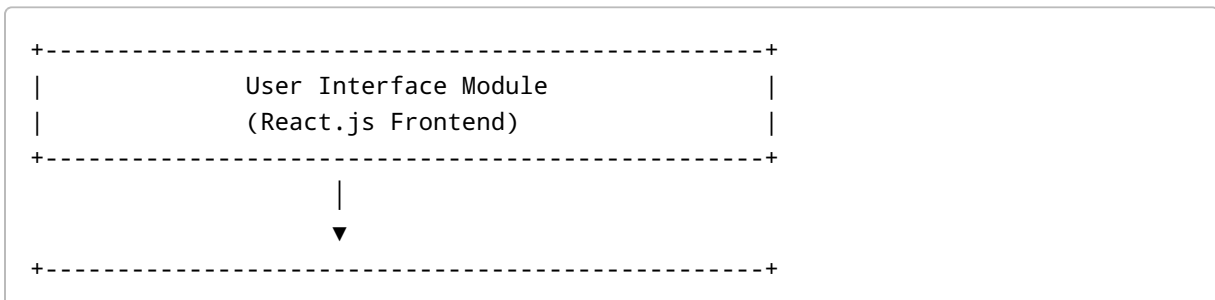
The high-level architecture illustrates how different components interact to process user queries and generate reliable responses.

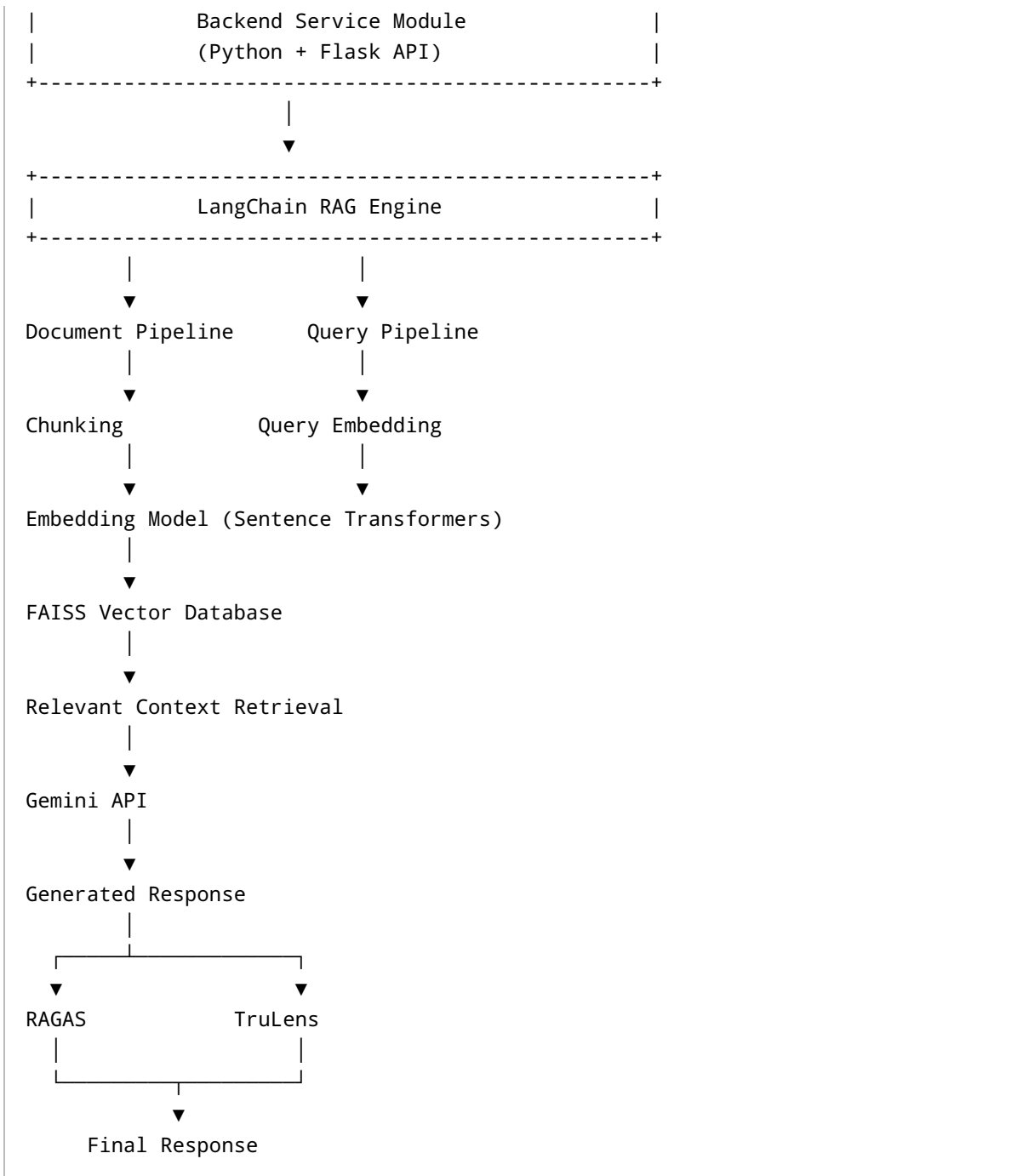




2. Module Diagram

The system is divided into independent modules, each responsible for a specific function.





3. System Components

The proposed architecture consists of the following major components.

3.1 User Interface

The frontend is developed using **React.js**. It allows users to upload documents, submit queries, and view AI-generated responses in an interactive interface.

Responsibilities

- User authentication (future enhancement)
 - Document upload
 - Question submission
 - Display AI responses
 - Display evaluation results
-

3.2 Backend Service

The backend is implemented using **Python** and the **Flask Framework**.

It acts as the communication layer between the frontend and the AI pipeline.

Responsibilities

- Receive user requests
 - Process uploaded files
 - Call LangChain services
 - Communicate with Gemini API
 - Return evaluated responses
-

3.3 Document Processing Module

Uploaded documents are preprocessed before being stored.

Processing includes:

- Text extraction
- Cleaning
- Normalization
- Metadata generation
- Recursive Character Text Chunking

The documents are divided into smaller chunks to improve semantic retrieval.

3.4 Embedding Generation Module

Each document chunk is converted into a semantic vector using the **Sentence Transformers (all-MiniLM-L6-v2)** embedding model.

Embeddings preserve the semantic meaning of text, enabling similarity-based retrieval rather than keyword matching.

3.5 Vector Database

The generated embeddings are stored in **FAISS (Facebook AI Similarity Search)**.

Responsibilities include:

- Vector storage
 - Semantic similarity search
 - Top-K document retrieval
 - Fast nearest-neighbor search
-

3.6 Retrieval Module

When a user submits a question:

1. The query is converted into an embedding.
2. FAISS searches for semantically similar document chunks.
3. The most relevant chunks are returned.

This retrieved context becomes the knowledge source for the LLM.

3.7 Large Language Model

The proposed system uses the **Gemini API**.

The model receives:

- User Question
- Retrieved Context

It generates a context-aware response grounded in the retrieved documents.

3.8 Evaluation Module

The generated response is evaluated using two complementary frameworks.

RAGAS

Evaluates:

- Faithfulness
- Context Precision
- Context Recall
- Answer Relevancy

Purpose:

To verify that the generated answer is supported by the retrieved documents and is free from hallucinations.

TruLens

Monitors:

- Prompt execution
- Retrieved context
- Generated response
- Latency
- Token usage
- API cost
- Feedback scores

Purpose:

To monitor the overall performance and reliability of the AI application.

4. Data Flow

The following sequence describes how data moves through the system.

Step 1

The user uploads one or more documents through the web interface.

↓

Step 2

The backend extracts and preprocesses the document text.



Step 3

Documents are divided into smaller chunks using Recursive Character Text Chunking.



Step 4

Each chunk is converted into vector embeddings using the Sentence Transformer embedding model.



Step 5

The generated embeddings are stored in the FAISS Vector Database.



Step 6

The user submits a natural language query.



Step 7

The query is converted into an embedding.



Step 8

FAISS retrieves the Top-K most relevant document chunks.



Step 9

The retrieved context and the user's question are provided to the Gemini API.

↓

Step 10

Gemini generates a context-aware response.

↓

Step 11

RAGAS evaluates retrieval quality, answer relevancy, and faithfulness.

↓

Step 12

TruLens records prompts, latency, token usage, evaluation scores, and execution logs.

↓

Step 13

The verified response is returned to the user.

5. Architecture Explanation

The proposed architecture follows the Retrieval-Augmented Generation (RAG) paradigm to improve the factual accuracy of AI-generated responses. Instead of relying solely on the internal knowledge of the LLM, the system retrieves relevant information from a trusted document repository before generating an answer.

Document chunking and semantic embeddings enable efficient retrieval of meaningful information. FAISS performs high-speed similarity search to identify the most relevant document chunks, which are then supplied to the Gemini LLM as contextual information.

To ensure response quality, the system integrates two evaluation frameworks. RAGAS evaluates the correctness and relevance of the generated response by measuring metrics such as Faithfulness, Context Precision, Context Recall, and Answer Relevancy. TruLens complements this by monitoring the entire application, recording prompts, retrieved context, response latency, API cost, and feedback metrics.

The modular design improves scalability and maintainability, allowing additional LLMs, embedding models, vector databases, or evaluation frameworks to be incorporated with minimal changes to the existing architecture. This makes the proposed system suitable for both academic research and real-world AI applications requiring reliable and explainable responses.

Conclusion

The proposed system architecture combines Retrieval-Augmented Generation with automated evaluation to create a reliable and trustworthy AI application. By integrating **Sentence Transformers** for embeddings, **FAISS** for semantic retrieval, **Gemini API** for response generation, **RAGAS** for response quality evaluation, and **TruLens** for application monitoring, the architecture effectively reduces hallucinations, improves retrieval accuracy, and ensures high-quality AI responses. The modular and scalable design also provides a strong foundation for future enhancements and production deployment.